



Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software
Ficha: 3134555

**Módulos integrados
(GA8-220501096-AA1-EV02)**

APRENDIZ
Johan Mauricio Sánchez Gaviria

INSTRUCTOR
Carlos Alberto Fuel

MARZO 2026

Documento de entrega formal del proyecto ALUMIPRO

Evidencia de integración y entrega del sistema web de gestión interna

INFORMACIÓN GENERAL

Proyecto: ALUMIPRO.

Tecnología: Java 17, Spring Boot, React, TypeScript, Vite y MySQL.

Desarrollador: Johan Mauricio Sánchez Gaviria

Fecha: 27 de marzo de 2026.

Versión entregada: 1.1.

El proyecto ALUMIPRO corresponde a un sistema web de gestión interna orientado a la administración de clientes, productos, ventas e identidades de acceso para una empresa del sector del aluminio. La interfaz web usa un enfoque basado en componentes y rutas protegidas, mientras que el backend concentra las reglas de negocio, la seguridad y la persistencia.

REQUERIMIENTOS DEL PROYECTO

Los requerimientos funcionales y no funcionales aprobados se consolidaron de acuerdo con la guía de aprendizaje, las clases de explicación de la evidencia y el estado funcional validado del sistema.

REQUERIMIENTOS FUNCIONALES APROBADOS

Código	Requerimiento	Descripción	Estado
RF01	Autenticación y autorización	Ingreso con usuarios de prueba y control de acceso por roles ADMIN y VENDEDOR.	Cumplido
RF02	Gestión de clientes	Consulta, creación,	Cumplido

		edición y eliminación de clientes desde interfaz y API.	
RF03	Gestión de productos	Consulta, creación, edición y eliminación de productos con control de stock; alta restringida al administrador.	Cumplido
RF04	Registro de ventas	Creación de ventas con selección de cliente, productos, cantidades y cálculo automático del total.	Cumplido
RF05	Regla de stock	Descuento automático del inventario e impedimento de venta cuando el stock es insuficiente.	Cumplido
RF06	Histórico y detalle	Consulta de ventas registradas y visualización de detalle por transacción.	Cumplido
RF07	Administración de usuarios	Gestión de usuarios y roles disponible únicamente para el perfil administrador.	Cumplido
RF08	Panel principal	Visualización de métricas básicas del negocio en el dashboard.	Cumplido

REQUERIMIENTOS NO FUNCIONALES APROBADOS

Código	Categoría	Descripción	Estado
RNF01	Seguridad	Autenticación con JWT, contraseñas cifradas y restricción por permisos.	Cumplido
RNF02	Usabilidad	Interfaz clara, menú	Cumplido

		lateral consistente, formularios en español y retroalimentación visual.	
RNF03	Mantenibilidad	Separación frontend/backend, organización por módulos, control de versiones y documentación.	Cumplido
RNF04	Portabilidad	Ejecución validada en Windows 11 con Java 17.0.12 LTS, Node 24.12.0 y XAMPP.	Cumplido
RNF05	Calidad	Ejecución satisfactoria de lint, pruebas automatizadas y build final en frontend; pruebas backend con Maven.	Cumplido
RNF06	Disponibilidad local	Despliegue local estable en http://localhost:5173 y API en http://localhost:8080/api .	Cumplido

INFORMACIÓN GENERAL DEL PROYECTO

Nombre del proyecto: ALUMIPRO Sistema web de gestión interna.

Lenguajes: Java 17 para backend y TypeScript para frontend.

Frameworks y librerías: Spring Boot para la API REST, React y Vite para la interfaz web, JWT para autenticación, MySQL para persistencia.

Repositorio GitHub: https://github.com/Johan-1998/alumipro_INC.git

URLs de ejecución: Aplicación web: <http://localhost:5173>. API REST: <http://localhost:8080/api>.

Entorno validado: Windows 11, IntelliJ IDEA o VS Code, XAMPP y navegador web moderno.

La arquitectura adoptada responde a una solución cliente-servidor. El frontend consume la API REST mediante peticiones HTTP y el backend centraliza la validación, el acceso a datos y las reglas de negocio. El uso de Spring Boot agiliza el desarrollo de servicios web en Java, mientras que React y Vite facilitan la construcción de interfaces modulares y compilación moderna.

ALUMIPRO - Arquitectura de integración

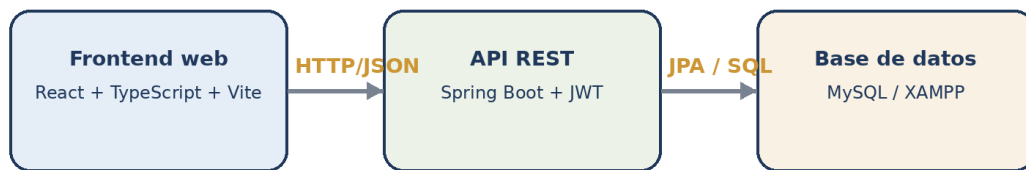


Figura 1. Arquitectura general de integración entre frontend, API REST y base de datos.

OBJETIVOS DEL PROYECTO

Objetivo general.

Desarrollar y entregar un sistema web integrado para la gestión interna de ALUMIPRO, con autenticación por roles, operación comercial básica y documentación técnica que soporte la instalación, validación y sustentación de la evidencia.

Objetivos específicos.

- Implementar autenticación segura con control de acceso diferenciado para administrador y vendedor.
- Gestionar clientes y productos mediante operaciones CRUD coherentes con las reglas del negocio.

- Registrar ventas con múltiples ítems, cálculo del total y validación de stock disponible.
- Consultar el histórico y el detalle de ventas para efectos de trazabilidad.
- Administrar usuarios del sistema desde el perfil de administrador.
- Documentar la arquitectura, el MER, los módulos, las pruebas y los archivos entregados.

MODELO ENTIDAD-RELACIÓN (MER)

El modelo de datos se estructuró sobre cinco entidades principales: Usuario, Cliente, Producto, Venta y Detalle_venta. La relación central del dominio se produce cuando un usuario vendedor registra una venta asociada a un cliente; cada venta contiene uno o varios detalles y cada detalle referencia un producto específico. Esta estructura favorece la normalización y la trazabilidad de la operación comercial.

ALUMIPRO - Modelo entidad-relación

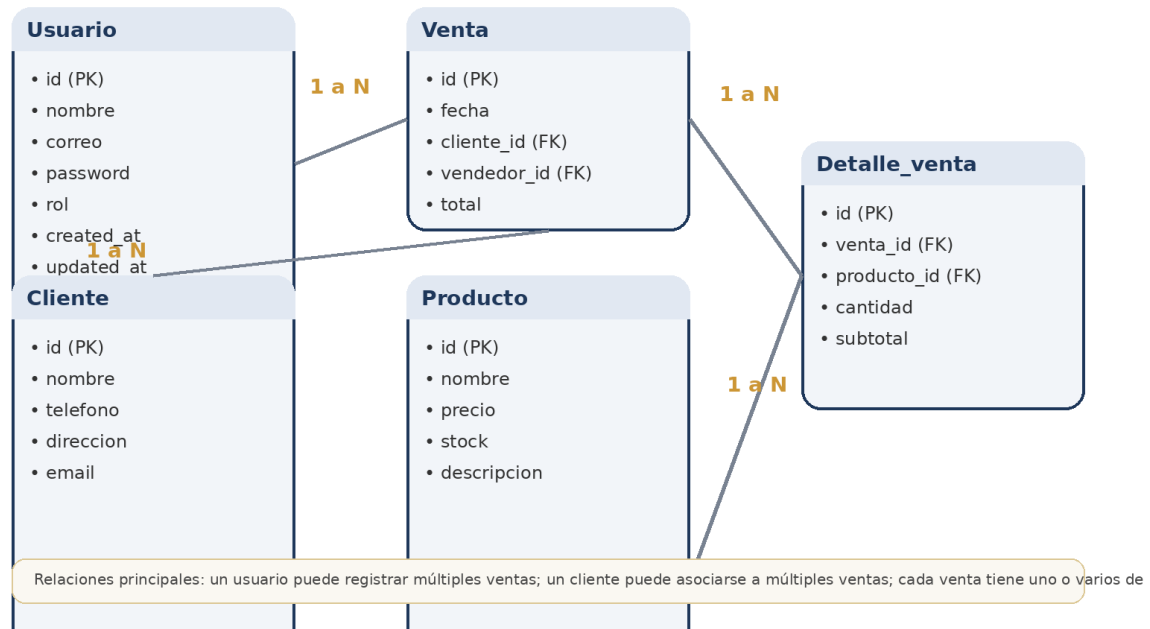


Figura 2. Modelo entidad-relación.

Entidad	Campos principales	Descripción funcional
Usuario	id, nombre, correo, password, rol, created_at, updated_at	Controla la identidad de acceso y define los permisos del sistema.
Cliente	id, nombre, telefono, direccion, email	Almacena los datos de los clientes que intervienen en ventas.
Producto	id, nombre, precio, stock, descripcion	Representa el inventario comercial disponible para la operación.
Venta	id, fecha, cliente_id, vendedor_id, total	Encabeza cada transacción comercial registrada por el sistema.
Detalle_venta	id, venta_id, producto_id, cantidad, subtotal	Descompone la venta en ítems para calcular cantidades y subtotales.

DESCRIPCIÓN DE MÓDULOS -BLOQUE TÉCNICO

En esta sección se incluyen extractos representativos del código y de la organización técnica que sustentan la integración solicitada en la evidencia.

ApiAuthController.java

```
@RestController
@RequestMapping("/api/auth")
public class ApiAuthController {
    @PostMapping("/login")
    public LoginResponse login(@RequestBody LoginRequest request) {
        // valida credenciales y devuelve token JWT
    }
}
```

El módulo de autenticación permite el ingreso con credenciales válidas y emite el token que habilita las rutas protegidas. En la sustentación se verificó el ingreso con el usuario administrador y con el usuario vendedor.

VentaService.java

```
@Transactional
public Integer registrarVenta(VentaForm form, Usuario vendedor) {
    // valida cliente, productos y cantidades
    // rechaza ítems con stock insuficiente
    // descuenta stock y registra total de la venta
}
```

La lógica de negocio del módulo de ventas garantiza que no se registren operaciones inconsistentes. La regla de stock insuficiente fue probada y el sistema rechazó correctamente la operación cuando la cantidad superó las existencias disponibles.

Protección de rutas en frontend

```
createBrowserRouter([
  { path: "/auth/login", element: <LoginPage /> },
  { path: "/", element: <AppLayout />, children: [ ... ] },
  { path: "/usuarios", element: <RequireAdmin><UsuariosPage /></RequireAdmin> },
])
```


El frontend restringe la navegación según el rol autenticado. El perfil vendedor no visualiza la sección Usuarios y no dispone de permisos para registrar productos, mientras que el perfil administrador sí puede gestionar ambos módulos.

DESCRIPCIÓN DE MÓDULOS, OPERACIÓN Y FLUJOS

Las capturas incluidas corresponden a la ejecución real del sistema.

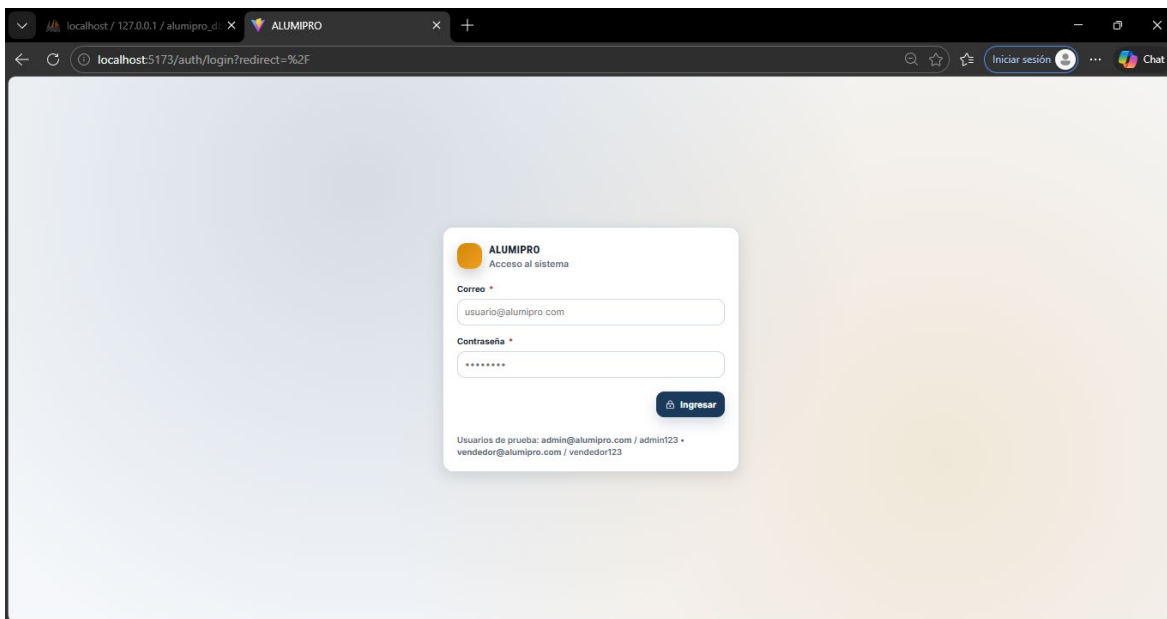


Figura 3. Pantalla de acceso con credenciales de prueba para administrador y vendedor.

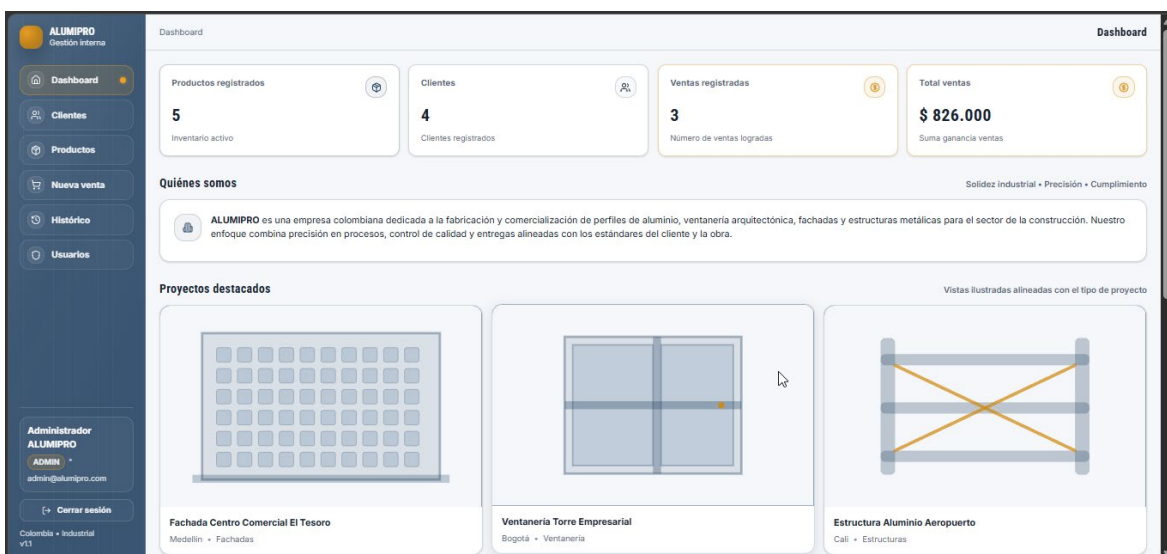


Figura 4. Dashboard principal con métricas resumidas y navegación lateral por módulos.

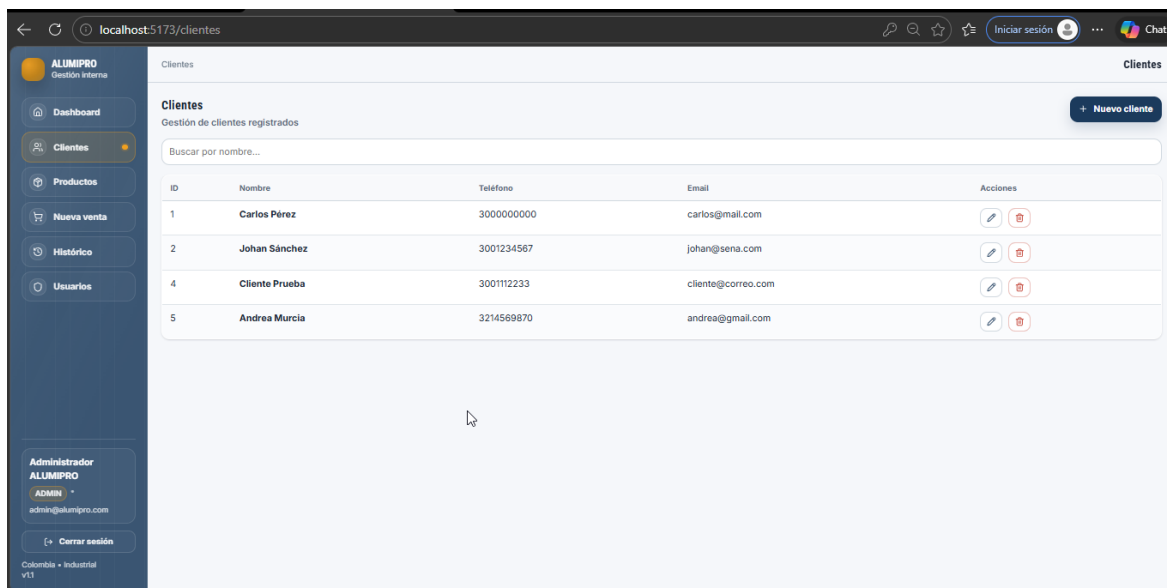


Figura 5. Módulo de clientes con operaciones de consulta y mantenimiento.

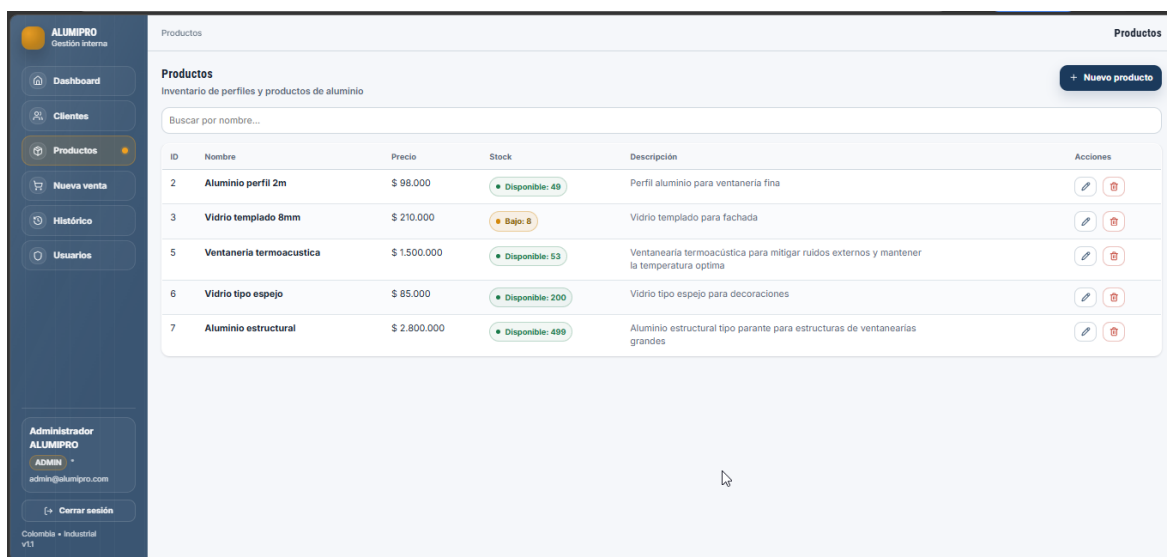


Figura 6. Módulo de productos con visualización de stock y acciones administrativas.

Registrar venta

Selecciona un cliente, agrega items y confirma el registro.

Cliente *
 Producto *

Cantidad *

Items de la venta Total: \$ 0

Sin items
Agrega productos para armar la venta.

Figura 7. Registro de venta con selección de cliente, producto, cantidad y consolidación del total.

Histórico de ventas Actualizar

Registro consolidado de ventas con detalle por transacción.

ID	Fecha	Cliente	Vendedor	Total	Detalle
14	23/3/2026	Andrea Murcia	—	\$ 630.000	Ver
13	23/3/2026	Johan Sánchez	—	\$ 98.000	Ver
12	18/3/2026	Carlos Pérez	—	\$ 98.000	Ver

Figura 8. Histórico de ventas con acceso al detalle por transacción.

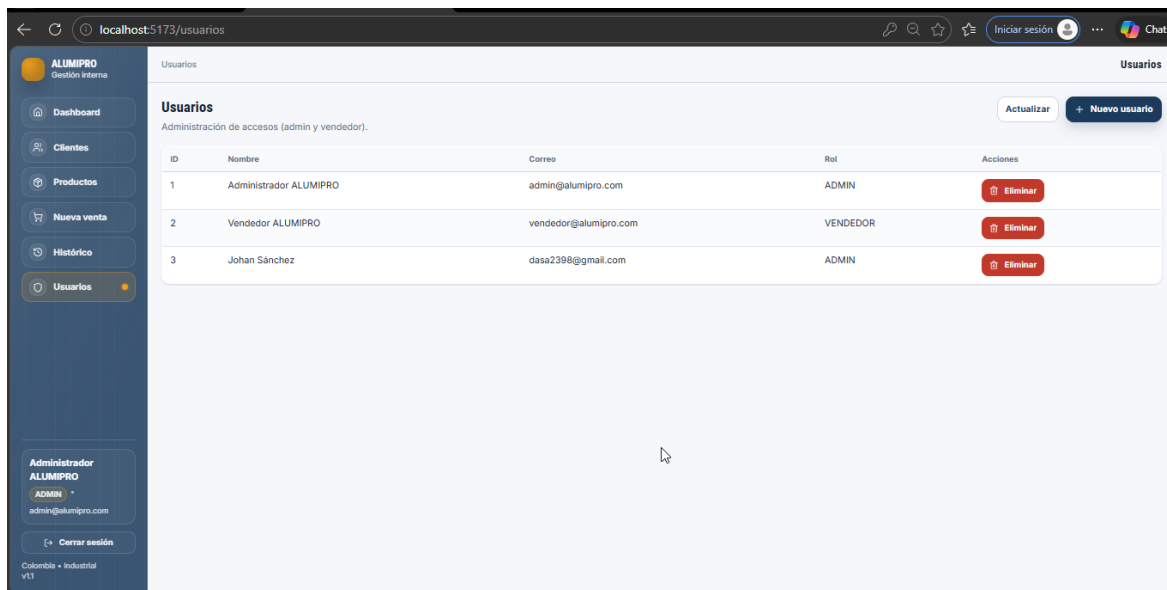


Figura 9. Administración de usuarios disponible para el perfil administrador.

Nota. Las capturas del formulario de nuevo cliente y del formulario de nuevo producto también fueron validadas durante la ejecución del sistema y pueden anexarse adicionalmente en sustentación. Asimismo, se verificó de manera funcional que el perfil vendedor no visualiza el módulo de usuarios y no cuenta con permisos para registrar productos.

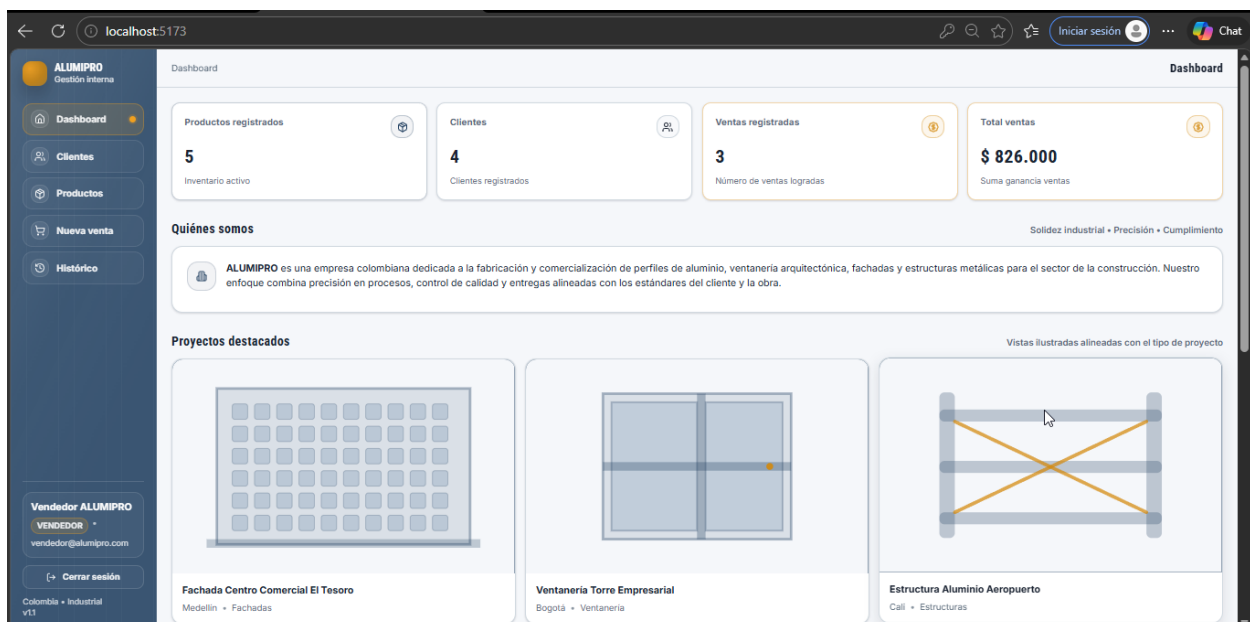


Figura 10. Dashboard con perfil vendedor

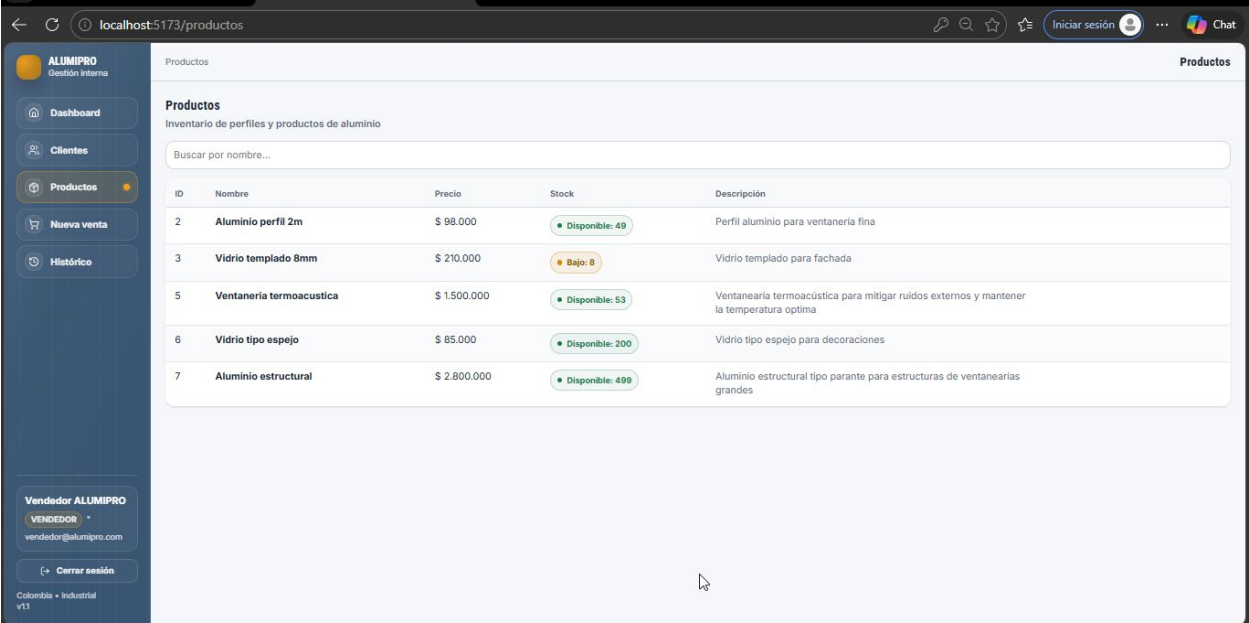


Figura 11. Perfil Vendedor ve los productos, pero no puede editar o añadir uno.

Matriz de roles y permisos

Módulo o acción	Administrador	Vendedor
Ingresar al sistema	Permitido	Permitido
Visualizar dashboard	Permitido	Permitido
Consultar clientes	Permitido	Permitido
Crear, editar y eliminar clientes	Permitido	Permitido
Consultar productos	Permitido	Permitido
Registrar productos	Permitido	No permitido
Editar o eliminar productos	Permitido	Restringido
Registrar venta	Permitido	Permitido
Consultar histórico	Permitido	Permitido
Administrar usuarios	Permitido	No permitido

PRUEBAS AUTOMATIZADAS

El proyecto se acompañó de pruebas automatizadas y de comandos de validación del entorno. En backend se usaron JUnit 5 y Mockito; en frontend se usaron herramientas de análisis estático, pruebas unitarias y compilación del proyecto para producción. Este enfoque mejora la mantenibilidad y la confianza sobre el sistema.

Componente	framework	Comando ejecutado	Resultado
Backend	JUnit 5, Mockito y Maven Surefire	mvn test	Ejecución satisfactoria en el entorno del proyecto.
Backend	Empaquetado Maven	mvn package	Compilación y empaquetado validados.
Frontend	ESLint	npm run lint	Sin errores de lint.
Frontend	Vitest pruebas unitarias	npm run test	Pruebas aprobadas.
Frontend	Vite TypeScript build	npm run build	Generación correcta del build de producción.

Credenciales de demostración y datos de prueba.

- Administrador: admin@alumipro.com / admin123.
- Vendedor: vendedor@alumipro.com / vendedor123.

PRUEBAS MANUALES DOCUMENTADAS

La siguiente matriz resume los casos funcionales más importantes validados durante la integración final. El resultado registrado corresponde al comportamiento observado en la ejecución del sistema.

Caso	Módulo	Escenario validado	Resultado esperado	Resultado
TM01	Autenticación	Ingreso exitoso con administrador.	Acceso al dashboard y visualización de Usuarios.	Cumplido
TM02	Autenticación	Ingreso exitoso con vendedor.	Acceso al dashboard sin módulo Usuarios.	Cumplido
TM03	Clientes	Creación, edición y eliminación de	Persistencia correcta y actualización en	Cumplido

		clientes.	listado.	
TM04	Productos	Creación, edición y eliminación de productos por administrador.	Persistencia correcta y actualización del stock.	Cumplido
TM05	Roles	Intento de gestión de usuarios por vendedor.	Acceso restringido.	Cumplido
TM06	Roles	Intento de registrar producto con vendedor.	Acción no disponible.	Cumplido
TM07	Ventas	Registro de venta con stock suficiente.	Venta creada y descuento de inventario.	Cumplido
TM08	Ventas	Registro de venta con stock insuficiente.	Operación rechazada y mensaje de validación.	Cumplido
TM09	Histórico	Consulta del histórico y acceso al detalle.	Listado visible y detalle por transacción.	Cumplido
TM10	Usuarios	Creación y administración de usuarios por administrador.	Operación disponible y consistente con el rol.	Cumplido

ARCHIVOS ENTREGADOS

La entrega se organiza como monorepo y se acompaña de evidencias de prueba, archivos de configuración y documento formal. La siguiente estructura resume los componentes principales entregados.

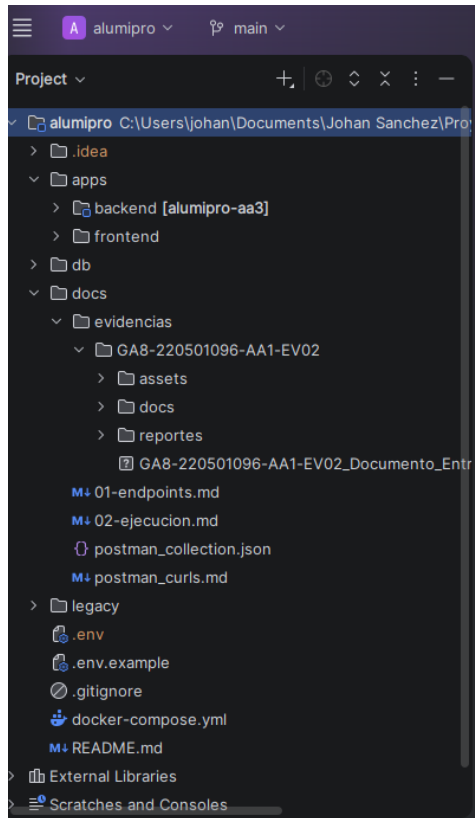


Figura 10. Estructura general del repositorio y de la carpeta de evidencia entregada.

Ruta o artefacto	Descripción
apps/backend/	Código fuente del backend Spring Boot, controladores, servicios, repositorios y pruebas.
apps/frontend/	Código fuente del frontend React, rutas protegidas, vistas, componentes y pruebas.
docs/evidencias/GA8-220501096-AA1-EV02/	Documento de entrega, anexos, capturas y reportes de validación.
.env.example y apps/frontend/.env.example	Variables de entorno de referencia para ejecutar el proyecto.
Repositorio GitHub	Histórico de versiones y respaldo del código fuente.

CONCLUSIONES

El proyecto ALUMIPRO se entregó como una solución web integrada y funcional, con separación clara entre frontend, backend y base de datos, y con trazabilidad sobre los módulos más relevantes del negocio.

Desde el área de calidad, el sistema quedó respaldado por validaciones manuales y automáticas, además de una documentación con la plantilla de entrega y con el estado real de la aplicación ejecutada.

REFERENCIAS

- Auth0. (n.d.). JSON Web Token introduction. <https://jwt.io/introduction>
- Meta Open Source. (n.d.). React documentation. <https://react.dev/>
- MySQL. (n.d.). MySQL reference manual. <https://dev.mysql.com/doc/>
- Oracle. (n.d.). Java Platform, Standard Edition 17 documentation.
<https://docs.oracle.com/en/java/javase/17/>
- OWASP Foundation. (n.d.). Authentication cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Spring. (n.d.). Spring Boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Vite. (n.d.). Vite guide. <https://vite.dev/guide/>